DeQL: A Decision Query Language for Prescriptive Analytics over Relational Data

MATTEO BRUCATO, OSM Data
FJODOR KHOLODKOV, OSM Data
SOREN LITTLE, OSM Data
JAKOB MAYER, OSM Data
DUC NGUYEN, OSM Data

We present DeQL (Decision Query Language), a SQL extension that adds first-class optimization constructs to relational query processing. `CREATE CANDIDATES` defines the decision domain from relational sources with a `DECISION KEY` specifying decision granularity. `DECIDE` declares typed decision variables, named constraints with grouping (`BY`) and filtering (`FILTER`), and an optimization objective over one or more candidate tables. The language provides a `SELECTION` modifier for subset selection, stochastic constraints for optimization under uncertainty, inline ML scoring via `PREDICT`, and quality modifiers (`WITHIN`, `TIMEOUT`) that let the optimizer trade quality for speed. This report gives the complete specification: formal grammar, design rationale, and end-to-end examples.

1 INTRODUCTION

DeQL (Decision Query Language) is a SQL extension that adds first-class decision constructs to relational query processing, enabling prescriptive analytics—computing optimal decisions subject to constraints—directly over organizational data. DeQL introduces two core statements:

- `CREATE CANDIDATES` defines the decision domain from relational sources, with a `DECISION KEY` specifying the granularity of decisions.
- `DECIDE` declares typed decision variables, named constraints with SQL-native grouping and filtering, and an optimization objective over candidate relations.

A `CREATE ABSTRACTION` statement prepares candidate data for abstraction-based decomposition, enabling scalable approximate solving.

The language is designed around composable primitives—typed decision variables (`BINARY`, `INTEGER`, `CONTINUOUS`), named constraints with `BY` and `FILTER` clauses, JOINs across candidate tables with different decision keys, and a `SELECTION` modifier that automates variable coupling for subset selection—rather than predefined problem classes. Existing work on in-database prescriptive analytics (SolveDB [8]), declarative package queries (PaQL [1]), and algebraic modeling languages (AMPL [4], Pyomo [6], JuMP [3]) addresses aspects of this space; DeQL integrates typed decision variables, SQL-native constraints, and relational output into a single language.

This report is a companion to the Decisionhouse vision paper, which presents the system architecture, Three-IR compilation pipeline, and research agenda. The present document focuses on the language itself: Section 2 presents design notes for key language constructs. Section 3 provides the language overview with syntax and worked examples. Section 4 gives the formal BNF grammar. Section 5 describes the execution model. Section 6 covers scope and extensions.

2 DESIGN NOTES

This section explains the key design decisions behind DeQL.

Authors' addresses: Matteo Brucato, OSM Data; Fjodor Kholodkov, OSM Data; Soren Little, OSM Data; Jakob Mayer, OSM Data; Duc Nguyen, OSM Data.

Table 1. SELECTION desugaring rules. Given a selection variable p and a companion variable x with bounds $[\ell, u]$, the system automatically generates the constraints shown.

Rule	User writes	System generates
Auto-multiply	SUM(col)	SUM(col * p)
Auto-link	x CONTINUOUS ...	$x \leq u \cdot p$
Auto-bound (lower)	BETWEEN ℓ AND u	$x \geq \ell \cdot p$
Auto-bound (upper)	BETWEEN ℓ AND u	$x \leq u \cdot p$
COUNT(*)	COUNT(*)	SUM(p)

2.1 The DECISION KEY

Every optimization problem over relational data selects or configures entities from a candidate set. The DECISION KEY identifies the *granularity* of the decision: which combination of columns defines a single decision entity.

Consider a shipping route optimization joining warehouses, stores, and routes. The FROM clause produces a flat join result, but should the system create one decision variable per warehouse, per store, or per route? Without explicit specification, the system cannot determine the variable structure from the schema alone. DECISION KEY resolves this ambiguity: DECISION KEY (wh_id, store_id) specifies that each unique warehouse-store pair is one decision entity.

The analogy to PRIMARY KEY is deliberate: just as PRIMARY KEY defines row identity for storage, DECISION KEY defines decision identity for optimization. The DECISION KEY determines how many decision variables the system instantiates, how constraints are grouped, and how results map back to relational tuples.

2.2 SELECTION Semantics

SELECTION is a modifier for DECISION COLUMNS that provides three automatic semantics, eliminating common optimization modeling boilerplate:

- **Auto-multiply.** Aggregates over data columns in the objective and constraints are automatically multiplied by the selection variable. For example, with selected SELECTION BINARY, writing SUM(price) in a constraint is logically rewritten to SUM(price * selected). This matches the intuition that unselected candidates contribute nothing.
- **Auto-link.** When the selection variable equals zero (entity not selected), all other decision variables for that entity are forced to zero. For example, if quantity CONTINUOUS BETWEEN 0 AND capacity accompanies active SELECTION BINARY, then active = 0 implies quantity = 0.
- **Auto-bound.** Declared type bounds apply only when the selection variable equals one (entity selected). The bound BETWEEN 0.01 AND 0.08 on a portfolio weight applies only to selected stocks; unselected stocks have weight exactly zero (from auto-link).

Table 1 summarizes the desugaring rules for SELECTION semantics. PaQL [1] introduced auto-multiply; DeQL extends the concept with auto-link and auto-bound. MAX COPIES provides a shorthand for bounded integer selection variables: selected SELECTION INTEGER MAX COPIES 5 allows each candidate to appear up to 5 times, replacing PaQL's REPEAT syntax with explicit variable semantics.

2.3 Composable Primitives

DeQL provides composable primitives: typed decision variables (BINARY, INTEGER, CONTINUOUS), constraint types (aggregate, grouped, filtered, disjunctive), and multi-granularity JOINs. Users

combine these freely to express problems that span multiple textbook categories—for example, a warehouse fulfillment problem may combine subset selection (which orders to fulfill), continuous allocation (how much inventory to allocate), and temporal sequencing (when to process each batch) in a single query.

The system infers exploitable structure from the resulting variable-constraint patterns. A query with binary variables and mutual-exclusivity constraints reveals assignment structure; a query with continuous flow variables and supply-demand balance reveals network flow structure. Structure detection operates on the query semantics, not on user-declared labels.

This design separates two concerns: the *user* specifies what to optimize (variables, constraints, objective); the *system* determines how to solve it (formulation, algorithm, decomposition).

2.4 Constraints as First-Class Citizens

Constraints in DeQL are named, grouped, and filtered—first-class query elements rather than opaque solver input.

Named constraints. Every constraint has a name (`CONSTRAINT supply_limit: ...`), enabling governance (which constraint was violated?), debugging (relax `supply_limit` and re-solve), and documentation.

Grouped constraints (`BY`). The `BY` clause generates one constraint per group, analogous to `GROUP BY` in SQL. Writing `SUM(quantity) BY wh_id <= capacity` generates one supply constraint per warehouse—the system determines the number of constraints from data cardinality.

Filtered constraints (`FILTER`). The `FILTER` clause restricts which candidates participate in a constraint, using SQL:2003 filter syntax. Writing `SUM(weight) <= 0.25 FILTER (WHERE sector = 'Technology')` applies the constraint only to technology-sector candidates. `BY` and `FILTER` compose naturally.

2.5 Multi-Granularity JOINs

Many real-world problems require decisions at multiple granularities—for example, assigning items to batches (item-level) while determining each batch's timing (batch-level). DeQL supports JOIN of candidate tables in the `DECIDE` query, where each candidate table retains its own `DECISION KEY`.

The key semantic distinction from SQL JOINs: decision variables are *not duplicated* when candidate tables are joined. If a batch appears in three join rows (one per assigned item), the batch's `start_time` variable is the same variable in all three rows—analogous to how a SQL JOIN links rows without merging table identities.

`FOREIGN DECISION KEY` declares referential relationships between candidate tables, analogous to `FOREIGN KEY` in SQL. This enables the system to validate join structure and detect decomposable subproblems.

3 LANGUAGE OVERVIEW

This section presents the complete DeQL syntax with query examples. Section 3.1 covers candidate definition; Section 3.2 covers the `DECIDE` statement; Section 3.3 details `SELECTION` desugaring; Section 3.4 covers constraint syntax; Section 3.5 covers multi-granularity JOINs; Section 3.6 covers schema-level constructs; Section 3.7 provides worked examples; Section 3.8 covers stochastic constraints; Section 3.9 covers quality modifiers; Section 3.10 discusses composability with SQL.

3.1 CREATE CANDIDATES

`CREATE CANDIDATES` defines the decision domain from relational sources. Unlike a table, it stores no new data; unlike a view, it carries decision-specific structure and metadata.

```
CREATE CANDIDATES <name>
DECISION KEY (<column_list>) AS
  <select_query>;
```

The `DECISION KEY` specifies which columns define a single decision entity. The system instantiates one set of decision variables per unique key value.

Example. Shipping routes as candidates:

```
CREATE CANDIDATES shipping_routes
DECISION KEY (wh_id, store_id) AS
  SELECT r.from_wh AS wh_id, r.to_store AS store_id,
         w.capacity, s.demand, r.cost, r.time_hrs
  FROM routes r
  JOIN warehouses w ON r.from_wh = w.wh_id
  JOIN stores s ON r.to_store = s.store_id;
```

The candidate definition accepts any SQL query supported by the underlying system. The physical representation—virtual, materialized in memory, or persisted—is managed by the system based on query complexity and reuse frequency.

An optional `DECISION COLUMNS` clause can be included directly in `CREATE CANDIDATES` to declare default decision variables at the schema level:

```
CREATE CANDIDATES shipping_routes
DECISION KEY (wh_id, store_id)
DECISION COLUMNS (
    active   SELECTION BINARY,
    quantity CONTINUOUS BETWEEN 0 AND capacity)
AS SELECT ...;
```

When `DECISION COLUMNS` is declared at the schema level, `DECIDE` queries can omit variable declarations and inherit them from the candidate definition.

3.2 The DECIDE Statement

The `DECIDE` statement is the core prescriptive query. Its anatomy:

```
DECIDE <result_name>
FROM <candidate_ref> [JOIN ...]
[DECISION COLUMNS (<variable_declarations>)]
[WHERE <filter_predicate>]
SUBJECT TO
  <constraint_list>
MINIMIZE | MAXIMIZE <objective_expression>;
```

- **`FROM`** references one or more named candidate sets (created by `CREATE CANDIDATES`).
- **`DECISION COLUMNS`** declares the decision variables for each candidate entity. Each variable has a name, type (`BINARY`, `INTEGER`, `CONTINUOUS`), optional `SELECTION` modifier, and optional bounds (`BETWEEN` ℓ `AND` u).
- **`WHERE`** filters candidates before optimization (predicate pushdown—candidates not satisfying the predicate are excluded from the decision domain).
- **`SUBJECT TO`** introduces the constraint block: a comma-separated list of named constraints.
- **`MINIMIZE`/`MAXIMIZE`** specifies the objective function as an aggregate expression over candidate columns and decision variables.

Example. Shipping plan with mixed variables:

```
DECIDE shipping_plan
FROM shipping_routes
```

```
DECISION COLUMNS (
    active   SELECTION BINARY,
    quantity CONTINUOUS BETWEEN 0 AND capacity)
WHERE time_hrs <= 48
SUBJECT TO
  CONSTRAINT supply_limit: SUM(quantity) BY wh_id <= capacity,
  CONSTRAINT meet_demand:  SUM(quantity) BY store_id = demand
MINIMIZE SUM(cost * quantity);
```

The `SELECTION BINARY` modifier on `active` provides auto-multiply, auto-link, and auto-bound semantics (Section 3.3).

3.3 SELECTION Semantics

The `SELECTION` modifier on a decision variable triggers three automatic transformations (desugaring rules detailed in Table 1):

Auto-multiply. Aggregates over data columns (columns from the candidate definition, not decision variables) are automatically multiplied by the selection variable. With `active SELECTION BINARY`:

- `SUM(cost)` in a constraint becomes `SUM(cost * active)`
- `COUNT(*)` becomes `SUM(active)`

Auto-link. When `active = 0`, all companion decision variables for that entity are forced to zero. For `quantity CONTINUOUS BETWEEN 0 AND capacity`, the system generates: $\texttt{quantity} \leq \texttt{capacity} \cdot \texttt{active}$.

Auto-bound. Lower bounds apply only when `active = 1`. For `weight CONTINUOUS BETWEEN 0.01 AND 0.08`, the system generates: $\texttt{weight} \geq 0.01 \cdot \texttt{active}$ and $\texttt{weight} \leq 0.08 \cdot \texttt{active}$.

MAX COPIES. For integer selection variables with bounded repetition:

```
DECISION COLUMNS (qty SELECTION INTEGER MAX COPIES 5)
```

This allows each candidate to appear 0 to 5 times, replacing PaQL's `REPEAT` syntax.

CREATE SELECTION CANDIDATES. A shorthand for candidates with an implicit `SELECTION BINARY` variable named `is_selected`:

```
CREATE SELECTION CANDIDATES products
DECISION KEY (product_id) AS
  SELECT product_id, price, weight FROM Inventory;
```

This is equivalent to creating candidates with `DECISION COLUMNS (is_selected SELECTION BINARY)`.

3.4 Constraint Syntax

All constraints follow the pattern:

```
CONSTRAINT <name>: <aggregate_expr> [BY <col_list>]
    <comparison_op> <bound_expr>
    [FILTER (WHERE <predicate>)]
```

Aggregate constraints. Standard SQL aggregates (`SUM`, `COUNT`, `MIN`, `MAX`, `AVG`) over expressions involving candidate columns and decision variables:

```
CONSTRAINT budget: SUM(price) <= 100
```

Grouped constraints (`BY`). The `BY` clause generates one constraint per group value, analogous to `GROUP BY`:

```
CONSTRAINT supply_limit: SUM(quantity) BY wh_id <= capacity
```

With 10 warehouses, this generates 10 constraints, each bounding the total quantity shipped from one warehouse.

Filtered constraints (FILTER). The FILTER clause restricts which candidates participate:

```
CONSTRAINT cap_tech: SUM(weight) <= 0.25
  FILTER (WHERE sector = 'Technology')
```

BY and FILTER compose: `SUM(qty) BY category <= limit FILTER (WHERE region = 'US')` generates one constraint per category, considering only US-region candidates.

Per-row constraints. Expressions without aggregates apply independently to each candidate:

```
CONSTRAINT deadline: start_time + processing_time <= deadline
```

Scheduling constraints. DISJUNCTIVE ensures non-overlapping resource usage:

```
CONSTRAINT no_overlap:
  DISJUNCTIVE(start_time, processing_time) BY machine
```

CUMULATIVE bounds aggregate resource consumption over time:

```
CONSTRAINT capacity:
  CUMULATIVE(start_time, processing_time, demand) <= max_capacity
```

3.5 Multi-Granularity

When a problem requires decisions at different granularities, multiple candidate tables with different DECISION KEYs are joined in the DECIDE query. Each candidate table retains its own key; decision variables are not duplicated across join rows. The following example assigns items to batches (one binary variable per item–batch pair) while scheduling each batch's start time (one continuous variable per batch):

```
CREATE CANDIDATES assignments
DECISION KEY (item_id, batch_id) AS
  SELECT i.item_id, b.batch_id, i.weight, b.capacity
  FROM items i CROSS JOIN batches b;

CREATE CANDIDATES timing
DECISION KEY (batch_id) AS
  SELECT batch_id, processing_time FROM batches;

DECIDE schedule
FROM assignments a JOIN timing t ON a.batch_id = t.batch_id
DECISION COLUMNS (
    assigned   SELECTION BINARY ON a,
    start_time CONTINUOUS BETWEEN 0 AND 1440 ON t)
SUBJECT TO
  CONSTRAINT one_batch: SUM(assigned) BY item_id = 1,
  CONSTRAINT batch_cap: SUM(weight * assigned) BY batch_id <= capacity
MINIMIZE MAX(start_time + processing_time);
```

Key semantics. The two candidate tables have different decision keys: assignments is keyed by (item_id, batch_id) and timing by (batch_id). When joined on batch_id, each batch's start_time variable appears in multiple join rows (one per assigned item) but refers to the same optimization variable.

ON clause in DECISION COLUMNS. When multiple candidate tables are joined, the ON clause specifies which candidate table a variable belongs to:

```
DECISION COLUMNS (
```

```
    assigned   SELECTION BINARY ON a,
    start_time CONTINUOUS BETWEEN 0 AND 1440 ON t)
```

FOREIGN DECISION KEY. Declares referential relationships between candidate tables at the schema level:

```
CREATE CANDIDATES item_batch_assignments
DECISION KEY (item_id, batch_id)
FOREIGN DECISION KEY (batch_id) REFERENCES batch_timing(batch_id)
AS SELECT ...;
```

This enables the system to validate join structure and detect decomposable subproblems.
Design principle. Decision variables must not appear in JOIN conditions—they belong in `SUBJECT TO`. JOIN links candidate tables by data columns; constraints link their decision variables.

3.6 Schema-Level Constructs

`CREATE CANDIDATES` can include schema-level `DECISION COLUMNS` and `CONSTRAINTS` blocks that are automatically enforced in any `DECIDE` query referencing the candidates.

```
CREATE CANDIDATES item_batch_assignments
DECISION KEY (item_id, batch_id)
FOREIGN DECISION KEY (batch_id)
  REFERENCES batch_timing(batch_id) AS t
DECISION COLUMNS (
  assigned SELECTION BINARY,
  batch_load = SUM(weight * assigned)
    REFERENCES batch_timing(batch_id))
CONSTRAINTS (
  one_batch: SUM(assigned) BY item_id = 1,
  batch_cap: batch_load <= capacity)
AS SELECT ...;
```

- **Free decision columns** (e.g., `assigned SELECTION BINARY`) declare variables the optimizer chooses.
- **Derived decision columns** use = syntax to define a variable whose value is determined by an aggregate over related rows. For example, `batch_load = SUM(weight * assigned)` with `REFERENCES batch_timing(batch_id)` creates a batch-level variable equal to the sum of assigned item weights. The `REFERENCES` clause specifies which foreign candidate table the derived variable belongs to.
- **`CONSTRAINTS`** declares structural constraints that are always enforced. Schema-level constraints are immutable: they cannot be overridden or relaxed in `DECIDE` queries. This separates concerns: optimization experts define structural constraints in the schema; business analysts add problem-specific constraints in `DECIDE` queries. For example, a workforce scheduling schema might enforce that no employee works more than one shift at a time—a physical impossibility, not a business preference.

3.7 Query Examples

3.7.1 Subset Selection (Knapsack). Select products for a shopping cart, maximizing total rating subject to budget and weight limits:

```
CREATE CANDIDATES products
DECISION KEY (product_id) AS
  SELECT product_id, price, weight, rating, category
  FROM Inventory WHERE in_stock = TRUE;
```

```
DECIDE cart
FROM products
DECISION COLUMNS (selected SELECTION BINARY)
SUBJECT TO
  CONSTRAINT budget: SUM(price) <= 100,
  CONSTRAINT capacity: SUM(weight) <= 50,
  CONSTRAINT per_category: SUM(weight) BY category <= 20
MAXIMIZE SUM(rating);
```

3.7.2 Continuous Allocation (Diet Problem). Determine servings of each food to meet nutritional requirements at minimum cost:

```
CREATE CANDIDATES foods
DECISION KEY (item_id) AS
  SELECT f.item_id, f.unit_cost, n.nutrient,
         n.amount, n.min_amount, n.max_amount
  FROM Foods f JOIN Nutrients n ON f.item_id = n.item_id;

DECIDE diet_plan
FROM foods
DECISION COLUMNS (servings CONTINUOUS BETWEEN 0 AND 10)
SUBJECT TO
  CONSTRAINT min_nutrient:
    SUM(amount * servings) BY nutrient >= min_amount,
  CONSTRAINT max_nutrient:
    SUM(amount * servings) BY nutrient <= max_amount
MINIMIZE SUM(unit_cost * servings);
```

3.7.3 Mixed Integer (Production Planning). Choose production quantities for each product to meet demand at minimum cost, with integer production levels:

```
CREATE CANDIDATES production_options
DECISION KEY (product_id) AS
  SELECT product_id, production_cost, demand,
         min_production, max_production
  FROM Products;

DECIDE production_plan
FROM production_options
DECISION COLUMNS (
    units_produced INTEGER
      BETWEEN min_production AND max_production)
SUBJECT TO
  CONSTRAINT meet_demand_c:
    units_produced >= demand
MINIMIZE SUM(production_cost * units_produced);
```

3.7.4 Assignment (Worker-Shift Matching). Assign workers to shifts such that each worker gets at most one shift and each shift has exactly one worker, minimizing skill mismatch:

```
CREATE CANDIDATES worker_shifts
DECISION KEY (worker_id, shift_id) AS
  SELECT w.worker_id, s.shift_id,
         w.skill_level, s.required_skill
  FROM Workers w CROSS JOIN Shifts s
  WHERE w.skill_level >= s.required_skill;
```

```
DECIDE schedule
FROM worker_shifts
SUBJECT TO
  CONSTRAINT one_shift:  COUNT(*) BY worker_id <= 1,
  CONSTRAINT one_worker: COUNT(*) BY shift_id = 1
MINIMIZE SUM(ABS(skill_level - required_skill));
```

The composite `DECISION KEY (worker_id, shift_id)` with mutual-exclusivity constraints reveals totally unimodular structure, enabling the system to select the Hungarian algorithm or network simplex instead of branch-and-bound.

3.7.5 Batch Scheduling (Multi-Granularity). Assign items to batches while determining each batch's start time. Two candidate tables with different `DECISION KEY`s:

```
CREATE CANDIDATES item_batch_assignments
DECISION KEY (item_id, batch_id) AS
  SELECT i.item_id, b.batch_id,
         i.weight, i.deadline, b.capacity
  FROM items i CROSS JOIN batches b;

CREATE CANDIDATES batch_timing
DECISION KEY (batch_id) AS
  SELECT batch_id, capacity, processing_time
  FROM batches;

DECIDE schedule
FROM item_batch_assignments a
  JOIN batch_timing t ON a.batch_id = t.batch_id
DECISION COLUMNS (
    assigned   SELECTION BINARY ON a,
    start_time CONTINUOUS BETWEEN 0 AND 1440 ON t)
SUBJECT TO
  CONSTRAINT one_batch:
    SUM(assigned) BY item_id = 1,
  CONSTRAINT batch_cap:
    SUM(weight * assigned) BY batch_id <= capacity,
  CONSTRAINT item_deadline:
    start_time + processing_time <= deadline,
  CONSTRAINT sequence:
    DISJUNCTIVE(start_time, processing_time) BY batch_id
MINIMIZE MAX(start_time + processing_time);
```

3.7.6 Budget Allocation with ML Scoring. Allocate marketing spend across products at discrete levels, where expected ROI is predicted by an ML model:

```
CREATE SELECTION CANDIDATES channels
DECISION KEY (product_id, spend_level)
CONSTRAINTS (
  one_level: COUNT(*) BY product_id = 1)
AS SELECT p.product_id, l.spend_level, l.spend,
         PREDICT(roi_model, p.product_id, l.spend_level)
           AS expected_roi
  FROM Products p
  CROSS JOIN SpendLevels l;

DECIDE spending_plan
FROM channels
```

```
SUBJECT TO
  CONSTRAINT total_budget: SUM(spend) <= 1000000
MAXIMIZE SUM(expected_roi);
```

The PREDICT function invokes a registered ML model inline. The SELECTION modifier on the implicit `is_selected` variable ensures auto-multiply applies to `SUM(spend)` and `SUM(expected_roi)`. The schema-level constraint `one_level` ensures exactly one spend level is selected per product.

3.7.7 Facility Location (Multi-Granularity). Decide which facilities to open (facility-level binary decision) and how much demand each facility serves for each customer (facility-customer-level continuous decision):

```
CREATE CANDIDATES facilities
DECISION KEY (facility_id) AS
  SELECT facility_id, open_cost, capacity
  FROM Facilities;

CREATE CANDIDATES facility_assignments
DECISION KEY (facility_id, customer_id) AS
  SELECT f.facility_id, c.customer_id,
         c.demand, t.transport_cost
  FROM Facilities f
  CROSS JOIN Customers c
  JOIN TransportCosts t
    ON t.facility_id = f.facility_id
    AND t.customer_id = c.customer_id;

DECIDE location_plan
FROM facilities f
  JOIN facility_assignments a
    ON f.facility_id = a.facility_id
DECISION COLUMNS (
    opened BINARY ON f,
    amount CONTINUOUS BETWEEN 0 AND demand ON a)
SUBJECT TO
  CONSTRAINT serve:
    SUM(amount) BY customer_id = demand,
  CONSTRAINT facility_cap:
    SUM(amount) BY facility_id <= capacity * opened
MINIMIZE SUM(open_cost * opened)
       + SUM(transport_cost * amount);
```

The opened variable is indexed by `facility_id`; the `amount` variable is indexed by `(facility_id, customer_id)`. The constraint `SUM(amount) BY facility_id <= capacity * opened` links the two levels: a closed facility serves no demand.

3.8 Stochastic Constraints

Following SPaQL [2], DeQL supports stochastic constraints for optimization under uncertainty. Uncertain data columns are declared with the `STOCHASTIC` function in `CREATE CANDIDATES`, specifying a distribution family and parameters drawn from base table columns:

```
CREATE CANDIDATES gpu_assignments
DECISION KEY (pool_id, workload_id) AS
  SELECT a.pool_id, a.workload_id,
         p.capacity, a.cost, a.latency_ms,
         STOCHASTIC('normal', w.demand_mean, w.demand_std)
           AS demand
```

```
  FROM assignments a
  JOIN gpu_pools p ON a.pool_id = p.pool_id
  JOIN workloads w ON a.workload_id = w.workload_id;
```

Constraints over stochastic columns use `WITH PROBABILITY` to specify chance constraints, and the objective can use `EXPECTED` to optimize in expectation:

```
DECIDE allocation_plan
FROM gpu_assignments
DECISION COLUMNS (
    active SELECTION BINARY,
    quantity CONTINUOUS BETWEEN 0 AND capacity)
WHERE latency_ms <= 200
SUBJECT TO
  CONSTRAINT pool_capacity:
    SUM(quantity) BY pool_id <= capacity,
  CONSTRAINT meet_demand:
    SUM(quantity) BY workload_id >= demand
    WITH PROBABILITY >= 0.95
MINIMIZE EXPECTED SUM(cost * quantity);
```

The `WITH PROBABILITY` clause on a constraint enforces it as a chance constraint: the system samples scenarios from the declared distributions and ensures the constraint holds with at least the specified probability. `EXPECTED` on the objective optimizes the expected value across scenarios.

3.9 Quality Modifiers

Quality modifiers `WITHIN` and `TIMEOUT` on the objective clause grant the decision planner permission to trade solution quality for speed. Two modifiers are supported:

```
MINIMIZE SUM(cost * quantity) WITHIN 5% TIMEOUT 30s
```

`WITHIN` X% guarantees the returned objective is within X% of optimal. `TIMEOUT` Xs requests the best feasible solution found within a time budget of X seconds. Both modifiers are optional and composable. They enable the decision planner to select abstraction-based decomposition or approximation algorithms when exact solving is impractical at scale.

3.10 Composability

`DECIDE` creates a standard table (analogous to `CREATE TABLE AS`). This enables three forms of composition:

SQL on results. Decision results can be queried, joined, and aggregated with standard SQL:

```
SELECT wh_id, SUM(quantity) AS total_shipped
FROM shipping_plan
GROUP BY wh_id;
```

Decision chaining. The output of one `DECIDE` becomes input to another enabling sequential decision-making:

```
CREATE CANDIDATES second_stage
DECISION KEY (store_id) AS
  SELECT * FROM shipping_plan WHERE quantity > 0;

DECIDE refined_plan
FROM second_stage ...;
```

CTE integration. `DECIDE` queries can appear in CTEs alongside SQL queries, enabling mixed descriptive-prescriptive workflows.

The relational closure property—every DeQL operator consumes and produces relations—mirrors SQL's composability and enables incremental adoption: organizations can integrate decision queries into existing SQL pipelines without architectural changes.

4 FORMAL GRAMMAR

This section presents the formal BNF grammar for DeQL. Terminal symbols are shown in UPPERCASE; non-terminals in ⟨*italic*⟩; optional elements in square brackets; alternatives separated by |; repetition with { }*.

4.1 CREATE CANDIDATES

```
<create_candidates_stmt> ::=
    CREATE [SELECTION] CANDIDATES <identifier>
    DECISION KEY ( <column_list> )
    [ FOREIGN DECISION KEY ( <column_list> )
      REFERENCES <identifier> ( <column_list> )
      [ AS <alias> ] ]
    [ DECISION COLUMNS ( <decision_decl_list> ) ]
    [ CONSTRAINTS ( <constraint_def_list> ) ]
    AS <select_query>

<column_list> ::=
    <identifier> { , <identifier> }*

<decision_decl_list> ::=
    <decision_decl> { , <decision_decl> }*

<decision_decl> ::=
    <identifier> [ SELECTION ] <var_type> [ <bounds> ]
    [ MAX COPIES <integer> ]
    [ ON <alias> ]
  | <identifier> = <aggregate_expr>
    [ REFERENCES <identifier> ( <column_list> ) ]

<var_type> ::=
    BINARY | INTEGER | CONTINUOUS

<bounds> ::=
    BETWEEN <expr> AND <expr>
```

When SELECTION precedes CANDIDATES in the statement, an implicit `is_selected SELECTION BINARY` variable is added to the decision columns.

4.2 DECIDE Statement

```
<decide_stmt> ::=
    DECIDE [ INTO ] <identifier>
    FROM <candidate_ref> { <join_clause> }*
    [ DECISION COLUMNS ( <decision_decl_list> ) ]
    [ WHERE <predicate> ]
    SUBJECT TO <constraint_list>
    <objective_clause>
```

```
<candidate_ref> ::=
    <identifier> [ <alias> ]

<join_clause> ::=
    JOIN <candidate_ref> ON <join_condition>

<objective_clause> ::=
    ( MINIMIZE | MAXIMIZE ) [ EXPECTED ] <aggregate_expr>
    [ WITHIN <number> % ]
    [ TIMEOUT <number> ( s | ms | m ) ]
```

4.3 Constraints

```
<constraint_list> ::=
    <constraint_def> { , <constraint_def> }*

<constraint_def> ::=
    CONSTRAINT <identifier> : <constraint_body>

<constraint_body> ::=
    <aggregate_constraint>
  | <per_row_constraint>
  | <scheduling_constraint>

<aggregate_constraint> ::=
    <aggregate_expr> [ BY <column_list> ]
    <comparison_op> <bound_expr>
    [ FILTER ( WHERE <predicate> ) ]
    [ WITH PROBABILITY <comparison_op> <number> ]

<per_row_constraint> ::=
    <scalar_expr> <comparison_op> <scalar_expr>

<scheduling_constraint> ::=
    DISJUNCTIVE ( <identifier> , <identifier> )
      [ BY <column_list> ]
  | CUMULATIVE ( <identifier> , <identifier> ,
      <identifier> ) <comparison_op> <expr>

<comparison_op> ::=
    <= | >= | = | < | >

<aggregate_expr> ::=
    SUM ( <expr> )
  | COUNT ( * )
  | MIN ( <expr> )
  | MAX ( <expr> )
  | AVG ( <expr> )

<bound_expr> ::=
    <literal> | <identifier> | <scalar_expr>
```

The BY clause in aggregate constraints generates one constraint instance per distinct value of the grouping columns, with the bound expression evaluated per group. The FILTER clause restricts which candidate rows participate in the aggregate.

4.4 Schema-Level Constructs

```
<constraint_def_list> ::=
    <schema_constraint> { , <schema_constraint> }*

<schema_constraint> ::=
    <identifier> : <constraint_body>
```

Schema-level DECISION COLUMNS and CONSTRAINTS are declared inside CREATE CANDIDATES and automatically enforced in any DECIDE query referencing those candidates. Decision columns may include derived columns using = syntax with an optional REFERENCES clause for cross-level aggregation in multi-granularity problems.

4.5 Stochastic Constructs

```
<stochastic_expr> ::=
    STOCHASTIC ( <string_literal> ,
      <expr> , <expr> )

<chance_constraint_modifier> ::=
    WITH PROBABILITY <comparison_op> <number>

<expected_objective> ::=
    ( MINIMIZE | MAXIMIZE ) EXPECTED <aggregate_expr>
```

The STOCHASTIC function declares an uncertain data column. The first argument is a distribution family ('normal', 'uniform'); the remaining arguments are distribution parameters drawn from base table columns. WITH PROBABILITY on a constraint enforces it as a chance constraint: the system samples scenarios and ensures the constraint holds with at least the specified probability. EXPECTED on the objective optimizes the expected value across sampled scenarios.

4.6 Prediction and AI Column Population

```
<predict_expr> ::=
    PREDICT ( <model_name> , <expr_list> )

<model_name> ::=
    <identifier>

<expr_list> ::=
    <expr> { , <expr> }*
```

The PREDICT function invokes a registered ML or AI model within a SELECT clause of CREATE CANDIDATES. The first argument names a model registered in the system catalog; subsequent arguments are column expressions passed as model inputs. The result is a scalar value that populates a data column participating in constraints and objectives. This covers both traditional ML model scoring (e.g., regression, classification) and LLM-based column population (e.g., sentiment analysis, entity extraction, compatibility scoring).

Table 2. Type combinations and their semantics. p denotes the selection variable; x denotes a companion variable.

Declaration	Domain	Semantics
`BINARY`	$\{0, 1\}$	Standalone binary decision
`INTEGER BETWEEN` ℓ `AND` u	$[\ell, u] \cap \mathbb{Z}$	Bounded integer
`CONTINUOUS BETWEEN` ℓ `AND` u	$[\ell, u]$	Bounded continuous
`SELECTION BINARY`	$\{0, 1\}$	Selection indicator with auto-multiply
`SELECTION INTEGER MAX COPIES` k	$\{0, \ldots, k\}$	Repetition count with auto-multiply
x `CONTINUOUS` + p `SELECTION`	$[0, u \cdot p]$	x linked to p via auto-link/auto-bound

4.7 Type System

Decision variables have three base types: `BINARY` ($\{0, 1\}$), `INTEGER` ($\mathbb{Z}$), and `CONTINUOUS` ($\mathbb{R}$). The `SELECTION` modifier is orthogonal to the base type and adds auto-multiply, auto-link, and auto-bound semantics.

Type checking rules:

- Objective and constraint expressions may be linear, quadratic, or nonlinear in decision variables. The decision planner selects an appropriate solver backend based on expression structure (e.g., LP/MILP for linear, QP/QCQP for quadratic, general-purpose nonlinear solvers for higher-order expressions).
- `SELECTION` variables must be `BINARY` or `INTEGER` (not `CONTINUOUS`).
- At most one `SELECTION` variable per candidate table.
- `BY` columns must reference data columns (not decision variables).
- Bounds in `BETWEEN` can reference data columns (evaluated per candidate row).

5 EXECUTION MODEL

A DeQL query flows through three intermediate representations before producing a result relation, following a Three-IR design.

The **parser** (a Rust-based SQL parser extension) tokenizes the query and produces **Abstract IR**—a normalized, symbolic, database-agnostic representation that preserves column references, symbolic expressions, constraint structure with grouping and filtering, and decision variable declarations. Abstract IR encodes no algorithm choice and binds no data.

The **decision planner** analyzes the Abstract IR to select an execution strategy, including structure detection (e.g., recognizing network flow or total unimodularity from variable-constraint patterns) and algorithm selection. The decision planner produces **Concrete IR**, which encodes the chosen mathematical formulation—still symbolic, using column references rather than actual values. Different formulations produce structurally different Concrete IRs from the same Abstract IR (e.g., generic MILP vs. LP relaxation vs. min-cost network flow).

The **materializer** fetches candidate data from the underlying sources and evaluates symbolic expressions against actual rows, producing **Materialized IR**: an objective coefficient vector, a sparse constraint matrix, variable bounds, and type annotations. This is what the solver receives. The **executor** dispatches the Materialized IR to a solver backend (HiGHS [7], Gurobi [5], or a specialized algorithm), and the resulting variable assignments are joined back with candidate metadata to produce the output relation.

The output is a standard relation that can be queried with SQL, joined with other tables, or fed as input to another `DECIDE` query.

6 SCOPE AND EXTENSIONS

6.1 Current Scope

DeQL can express:

- **Linear and mixed-integer programs**: subset selection, resource allocation, assignment, and scheduling via `BINARY`, `INTEGER`, and `CONTINUOUS` decision variables with `SELECTION` desugaring.
- **Quadratic objectives and constraints**: portfolio optimization with covariance terms, budget allocation with cannibalization penalties.
- **Nonlinear expressions**: routed to appropriate solver backends (e.g., IPOPT, BARON). The language imposes no restriction on expression linearity; the decision planner matches expression structure to solver capabilities, just as a SQL optimizer matches query structure to access paths.
- **Constraint programming constructs**: `DISJUNCTIVE` and `CUMULATIVE` for scheduling problems (Section 3.4).
- **Stochastic constraints**: `STOCHASTIC` declares uncertain data columns with distribution parameters; `WITH PROBABILITY` enforces chance constraints; `EXPECTED` optimizes in expectation (Section 3.8), following SPaQL [2].
- **Inline ML scoring**: `PREDICT` invokes registered models to populate data columns during candidate construction (Section 3.7.6).
- **AI-powered column population**: LLM and ML model calls within `CREATE CANDIDATES` queries populate derived columns—sentiment scores, demand forecasts, compatibility ratings—that participate in constraints and objectives, following the same pattern as `PREDICT` in BigQuery ML and similar systems.
- **Quality modifiers**: `WITHIN` and `TIMEOUT` on the objective clause grant the decision planner permission to trade solution quality for speed via abstraction-based decomposition (Section 3.9).

6.2 Limitations and Future Work

- **Multi-stage stochastic optimization.** Problems where decisions alternate with uncertain observations (e.g., decide inventory levels, observe demand, adjust orders) require a recourse or dynamic programming framework beyond the current single-stage model. Deterministic sequential decisions are already supported via decision chaining (Section 3.10).
- **Lazy constraint generation.** Problems where the constraint set is exponential and constraints are generated on demand during solving (e.g., DFJ subtour elimination for TSP, cutting planes) require solver callbacks that the current language does not expose. Compact formulations (e.g., MTZ for TSP) remain expressible.
- **Multi-objective optimization.** The current language supports a single objective. An extension such as `MINIMIZE (cost, -profit) PARETO` would return the Pareto frontier as a set of non-dominated solutions, each materialized as a separate result relation.

REFERENCES

[1] Matteo Brucato, Azza Abouzied, and Alexandra Meliou. 2018. Package Queries: Efficient and Scalable Computation of High-Order Constraints. *The VLDB Journal* 27, 5 (2018), 693–718.

[2] Matteo Brucato, Nishant Yadav, Azza Abouzied, Peter J. Haas, and Alexandra Meliou. 2020. Stochastic Package Queries in Probabilistic Databases. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020*. ACM, 269–283. https://doi.org/10.1145/3318464.3389765

[3] Iain Dunning, Joey Huchette, and Miles Lubin. 2017. JuMP: A Modeling Language for Mathematical Optimization. *SIAM Rev.* 59, 2 (2017), 295–320.

[4] Robert Fourer, David M. Gay, and Brian W. Kernighan. 2002. *AMPL: A Modeling Language for Mathematical Programming* (2nd ed.). Cengage Learning.

[5] Gurobi Optimization, LLC. 2024. Gurobi Optimizer Reference Manual. https://www.gurobi.com

[6] William E. Hart, Jean-Paul Watson, and David L. Woodruff. 2011. Pyomo: modeling and solving mathematical programs in Python. *Mathematical Programming Computation* 3, 3 (2011), 219–260.

[7] Qi Huangfu and J. A. Julian Hall. 2018. Parallelizing the dual revised simplex method. *Mathematical Programming Computation* 10, 1 (2018), 119–142.

[8] Laurynas Siksnys and Torben Bach Pedersen. 2016. SolveDB: Integrating Optimization Problem Solvers Into SQL Databases. In *Proceedings of the 28th International Conference on Scientific and Statistical Database Management*. ACM, 14:1–14:12.